

ATIVIDADE AVALIATIVA 4 - OPENGL

EXERCÍCIO OPENGL - ILUMINAÇÃO

Ana Vitória Cardoso Queiroz - 172706

Maria Clara Muharem - 266864

Projeto desenvolvido em OpenGL utilizando iluminação e texturas para simular um cinema 3D.

Foram utilizadas fontes de luz ambiente, iluminação lateral e iluminação superior para gerar contraste e profundidade na cena.

Bibliotecas utilizadas:

- OpenGL (GL/glut.h)
- FreeGLUT / GLUT
- stdio.h
- math.h
- string.h
- stdlib.h

Compilação: gcc main.c camera.c image.c obj.c textura.c -o main -lfreeglut -lopengl32 -lglu32

Códigos modificados pela iluminação:

Main.c

C/C++

```
// Ana Vitória Cardoso Queiroz - 172706
```

```
// Maria Clara Muharem - 266864
```

```
#include <GL/glut.h>
```

```
#include <stdio.h>
```

```
#include "camera.h"
```

```
#include "obj.h"
```

```
#include "textura.h"
```

```
void display()
```

```
{
```

```
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

```
    glLoadIdentity();
```

```
    gluLookAt(0, 5, distancia,
```

```
             0, 3.5, 0,
```

```
             0, 1, 0);
```

```
    glRotatef(anguloX, 1, 0, 0);
```

```
    glRotatef(anguloY, 0, 1, 0);
```

```
// posições das luzes
```

```
GLfloat pos0[] = {0.0, 8.2, 0.0, 1.0};
```

```
glLightfv(GL_LIGHT0, GL_POSITION, pos0);
```

```
static float zLuz = -20.0;
```

```

zLuz += 0.3;
if (zLuz > 28) zLuz = -22;
GLfloat pos1[] = {14.5, 3.8, zLuz, 1.0};
glLightfv(GL_LIGHT1, GL_POSITION, pos1);

GLfloat pos2[] = {0.0, 5.2, -8.0, 1.0};
GLfloat dir2[] = {0.0, -0.35, -0.94};
glLightfv(GL_LIGHT2, GL_POSITION, pos2);
glLightfv(GL_LIGHT2, GL_SPOT_DIRECTION, dir2);

// cena
desenharChaoComTextura();
desenharTeto();
desenharParedesComTextura();
desenharCorredorLateral();
desenharEscadaria();
desenharPorta();
desenharTela();
desenharPlateia();
desenharLuzes();
desenharCaixasSom();

glutSwapBuffers();
}

void init()
{
    glEnable(GL_NORMALIZE);
    glEnable(GL_DEPTH_TEST);
    glClearColor(0.05, 0.05, 0.1, 1.0);
    carregarTexturas();

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(55, 1.0, 0.8, 60.0);
    glMatrixMode(GL_MODELVIEW);

    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    glEnable(GL_LIGHT1);
    glEnable(GL_LIGHT2);
    glEnable(GL_COLOR_MATERIAL);

    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);

    GLfloat luz0_amb[] = {0.25, 0.22, 0.22, 1.0};
    GLfloat luz0_dif[] = {0.85, 0.80, 0.85, 1.0};
    GLfloat luz0_esp[] = {0.55, 0.55, 0.55, 1.0};

    glLightfv(GL_LIGHT0, GL_AMBIENT, luz0_amb);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, luz0_dif);
    glLightfv(GL_LIGHT0, GL_SPECULAR, luz0_esp);
    glLightf(GL_LIGHT0, GL_LINEAR_ATTENUATION, 0.03);
    glLightf(GL_LIGHT0, GL_QUADRATIC_ATTENUATION, 0.005);

    GLfloat luz1_amb[] = {0.05, 0.03, 0.01, 1.0};
    GLfloat luz1_dif[] = {1.2, 0.95, 0.55, 1.0};
    GLfloat luz1_esp[] = {0.8, 0.7, 0.5, 1.0};

    glLightfv(GL_LIGHT1, GL_AMBIENT, luz1_amb);
    glLightfv(GL_LIGHT1, GL_DIFFUSE, luz1_dif);
    glLightfv(GL_LIGHT1, GL_SPECULAR, luz1_esp);
    glLightf(GL_LIGHT1, GL_LINEAR_ATTENUATION, 0.08);
    glLightf(GL_LIGHT1, GL_QUADRATIC_ATTENUATION, 0.02);

    GLfloat luz2_amb[] = {0.0, 0.0, 0.0, 1.0};
    GLfloat luz2_dif[] = {1.3, 1.2, 1.0, 1.0};
    GLfloat luz2_esp[] = {1.0, 0.95, 0.85, 1.0};

```

```

glLightfv(GL_LIGHT2, GL_AMBIENT, luz2_amb);
glLightfv(GL_LIGHT2, GL_DIFFUSE, luz2_dif);
glLightfv(GL_LIGHT2, GL_SPECULAR, luz2_esp);
glLightf(GL_LIGHT2, GL_LINEAR_ATTENUATION, 0.05);
glLightf(GL_LIGHT2, GL_QUADRATIC_ATTENUATION, 0.01);

glLightf(GL_LIGHT2, GL_SPOT_CUTOFF, 35.0);
glLightf(GL_LIGHT2, GL_SPOT_EXPONENT, 12.0);

GLfloat luz_global[] = {0.18, 0.16, 0.20, 1.0};
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, luz_global);

GLfloat mat_spec[] = {0.85, 0.80, 0.75, 1.0};
GLfloat brilho[] = {60.0};
glMaterialfv(GL_FRONT, GL_SPECULAR, mat_spec);
glMaterialfv(GL_FRONT, GL_SHININESS, brilho);
}

int main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(1024, 768);
    glutCreateWindow("Cinema 3D - MC.AV");

    init();

    glutDisplayFunc(display);
    glutMouseFunc(controlarCameraMouse);
    glutMotionFunc(controlarCameraMotion);
    glutKeyboardFunc(controlarCameraTeclado);
    glutSpecialFunc(controlarCameraEspecial);

    glutMainLoop();
    return 0;
}

```

obj.c

```

C/C++
// Ana Vitória Cardoso Queiroz - 172706
// Maria Clara Muharem - 266864

#include "obj.h"
#include <GL/glut.h>
#include <math.h>

void desenharCadeiraVIP()
{
    glPushMatrix();

    // base
    glColor3f(0.85, 0.72, 0.35);
    glPushMatrix();
    glTranslatef(0, 0.08, 0);
    glScalef(1.8, 0.08, 1.6);
    glNormal3f(0, 1, 0);
    glutSolidCube(1.0);
    glPopMatrix();

    // assento
    glColor3f(0.45, 0.25, 0.18);
    glPushMatrix();

```

```

glTranslatef(0, 0.35, 0.1);
glScalef(1.7, 0.25, 1.5);
glNormal3f(0, 1, 0);
glutSolidCube(1.0);
glPopMatrix();

// assento acabamento
glColor3f(0.55, 0.18, 0.22);
glPushMatrix();
glTranslatef(0, 0.5, 0.1);
glScalef(1.65, 0.1, 1.45);
glNormal3f(0, 1, 0);
glutSolidCube(1.0);
glPopMatrix();

// encosto
glColor3f(0.45, 0.25, 0.18);
glPushMatrix();
glTranslatef(0, 0.9, 0.6);
glRotatef(18, 1, 0, 0);
glScalef(1.7, 1.3, 0.28);

// Normal para face inclinada
glNormal3f(0, 0.95, 0.31);
glutSolidCube(1.0);
glPopMatrix();

// encosto
glColor3f(0.55, 0.18, 0.22);
glPushMatrix();
glTranslatef(0, 0.9, 0.58);
glRotatef(18, 1, 0, 0);
glScalef(1.65, 1.25, 0.1);
glNormal3f(0, 0.95, 0.31);
glutSolidCube(1.0);
glPopMatrix();

// apoio cabeca
glColor3f(0.45, 0.25, 0.18);
glPushMatrix();
glTranslatef(0, 1.45, 0.75);
glRotatef(10, 1, 0, 0);
glScalef(1.2, 0.35, 0.2);
glNormal3f(0, 0.98, 0.17);
glutSolidCube(1.0);
glPopMatrix();

// apoio braço esquerdo
glColor3f(0.35, 0.18, 0.08);
glPushMatrix();
glTranslatef(-0.95, 0.55, 0.05);
glScalef(0.18, 0.35, 1.5);
glNormal3f(-1, 0, 0);
glutSolidCube(1.0);
glPopMatrix();

// apoio braço esquerdo
glColor3f(0.55, 0.18, 0.22);
glPushMatrix();
glTranslatef(-0.95, 0.75, 0.05);
glScalef(0.18, 0.12, 1.45);
glNormal3f(-1, 0, 0);
glutSolidCube(1.0);
glPopMatrix();

// apoio braço direito (base)
glColor3f(0.35, 0.18, 0.08);
glPushMatrix();

```

```

glTranslatef(0.95, 0.55, 0.05);
glScalef(0.18, 0.35, 1.5);
glNormal3f(1, 0, 0);
glutSolidCube(1.0);
glPopMatrix();

// apoio braço direito (acabamento)
glColor3f(0.55, 0.18, 0.22);
glPushMatrix();
glTranslatef(0.95, 0.75, 0.05);
glScalef(0.18, 0.12, 1.45);
glNormal3f(1, 0, 0);
glutSolidCube(1.0);
glPopMatrix();
}

void desenharDegrau(float zPos, float yPos, float largura)
{
    // base do degrau
    glColor3f(0.22, 0.22, 0.28);
    glPushMatrix();
    glTranslatef(0, yPos - 0.1, zPos);
    glScalef(largura, 0.3, 3.2);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();

    // piso do degrau
    glColor3f(0.5, 0.08, 0.15);
    glPushMatrix();
    glTranslatef(0, yPos + 0.05, zPos);
    glScalef(largura - 0.5, 0.05, 3.0);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();

    // borda dourada
    glColor3f(0.85, 0.72, 0.35);
    glPushMatrix();
    glTranslatef(0, yPos + 0.18, zPos + 1.55);
    glScalef(largura - 0.8, 0.03, 0.08);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();
}

void desenharPlateia()
{
    int fileiras = 8;
    float zInicio = -3.0;
    float espacamentoZ = 3.8;
    float alturaPorDegrau = 0.65;

    for (int i = 0; i < fileiras; i++)
    {
        float zPos = zInicio + (i * espacamentoZ);
        float yPos = i * alturaPorDegrau;

        float larguraEsquerda = 9.5;
        desenharDegrau(zPos, yPos, larguraEsquerda);

        glPushMatrix();
        glTranslatef(11.5, 0, 0);
        desenharDegrau(zPos, yPos, larguraEsquerda);
        glPopMatrix();

        for (float x = -7.5; x <= -2.5; x += 2.2)
        {

```

```

        glPushMatrix();
        glTranslatef(x, yPos + 0.25, zPos);
        desenharCadeiraVIP();
        glPopMatrix();
    }

    for (float x = 3.5; x <= 8.5; x += 2.2)
    {
        glPushMatrix();
        glTranslatef(x, yPos + 0.25, zPos);
        desenharCadeiraVIP();
        glPopMatrix();
    }

    if (i >= 5)
    {
        for (float x = -10; x <= -8; x += 2.0)
        {
            glPushMatrix();
            glTranslatef(x, yPos + 0.25, zPos);
            desenharCadeiraVIP();
            glPopMatrix();
        }
        for (float x = 9; x <= 11; x += 2.0)
        {
            glPushMatrix();
            glTranslatef(x, yPos + 0.25, zPos);
            desenharCadeiraVIP();
            glPopMatrix();
        }
    }
}

```

```

void desenharChao()
{
    glColor3f(0.22, 0.22, 0.28);
    glPushMatrix();
    glTranslatef(0, -0.3, 0);
    glScalef(32, 0.1, 55);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.5, 0.08, 0.15);
    glPushMatrix();
    glTranslatef(0, -0.25, 0);
    glScalef(30, 0.05, 53);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();
}

```

```

void desenharTeto()
{
    glColor3f(0.08, 0.06, 0.12);
    glPushMatrix();
    glTranslatef(0, 9.0, 0);
    glScalef(32, 0.15, 55);
    glNormal3f(0, -1, 0);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.85, 0.72, 0.35);
    for (float x = -15; x <= 15; x += 30)
    {
        glPushMatrix();
        glTranslatef(x, 8.6, 0);
    }
}

```

```

    glScalef(0.3, 0.4, 54);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();
}

glColor3f(1.0, 0.9, 0.6);

for (float x = -11; x <= -3; x += 2.0)
{
    for (float z = -20; z <= 25; z += 4.0)
    {
        glPushMatrix();
        glTranslatef(x, 8.92, z);
        glScalef(0.8, 0.03, 1.2);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();
    }
}

for (float x = 4; x <= 12; x += 2.0)
{
    for (float z = -20; z <= 25; z += 4.0)
    {
        glPushMatrix();
        glTranslatef(x, 8.92, z);
        glScalef(0.8, 0.03, 1.2);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();
    }
}

for (float z = -18; z <= 27; z += 3.5)
{
    glPushMatrix();
    glTranslatef(0, 8.92, z);
    glScalef(1.5, 0.03, 1.0);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();
}

for (float z = -18; z <= 27; z += 2.5)
{
    glPushMatrix();
    glTranslatef(11, 8.92, z);
    glScalef(0.6, 0.03, 1.8);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();
}

for (float x = -11; x <= -3; x += 2.0)
{
    for (float z = -20; z <= 25; z += 4.0)
    {
        glPushMatrix();
        glTranslatef(x, 8.89, z);
        glScalef(0.9, 0.02, 1.3);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();
    }
}

for (float x = 4; x <= 12; x += 2.0)

```

```

{
    for (float z = -20; z <= 25; z += 4.0)
    {
        glPushMatrix();
        glTranslatef(x, 8.89, z);
        glScalef(0.9, 0.02, 1.3);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();
    }
}

// projetor
glColor3f(0.3, 0.3, 0.35);
glPushMatrix();
glTranslatef(0, 8.2, -15);
glScalef(1.5, 0.3, 1.8);
glNormal3f(0, 1, 0);
glutSolidCube(1);
glPopMatrix();

glColor3f(0.1, 0.1, 0.15);
glPushMatrix();
glTranslatef(0, 8.0, -14.5);
glScalef(1.2, 0.2, 0.8);
glNormal3f(0, 1, 0);
glutSolidCube(1);
glPopMatrix();

glColor3f(0.5, 0.5, 0.6);
glPushMatrix();
glTranslatef(0, 7.9, -14.8);
glScalef(0.8, 0.1, 0.5);
glNormal3f(0, 1, 0);
glutSolidCube(1);
glPopMatrix();
}

void desenharTela()
{
    // moldura externa
    glColor3f(0.85, 0.72, 0.35);
    glPushMatrix();
    glTranslatef(0, 5.0, -23.5);
    glScalef(12, 8.0, 0.1);
    glutSolidCube(1);
    glPopMatrix();

    GLfloat emissive[] = {0.2, 0.2, 0.2, 1.0}; // leve brilho
    glMaterialfv(GL_FRONT, GL_EMISSION, emissive);

    glColor3f(0.95, 0.95, 0.95);
    glPushMatrix();
    glTranslatef(0, 5.0, -23.3);
    glScalef(11.5, 7.5, 0.05);
    glutSolidCube(1);
    glPopMatrix();

    // desliga emissão
    GLfloat no_emissive[] = {0.0, 0.0, 0.0, 1.0};
    glMaterialfv(GL_FRONT, GL_EMISSION, no_emissive);

    // laterais
    glColor3f(0.65, 0.12, 0.15);
    glPushMatrix();
    glTranslatef(-6.5, 5.0, -23);
    glScalef(1.2, 8.0, 0.4);
    glutSolidCube(1);
}

```



```

glPopMatrix();

glPushMatrix();
glTranslatef(6.5, 5.0, -23);
glScalef(1.2, 8.0, 0.4);
glutSolidCube(1);
glPopMatrix();

// topo
glPushMatrix();
glTranslatef(0, 8.8, -23);
glScalef(13, 0.8, 0.5);
glutSolidCube(1);
glPopMatrix();
}

void desenharLuzes()
{
    for (float z = -20; z <= 25; z += 3.5)
    {

        glColor3f(0.6, 0.5, 0.3);

        // esquerda
        glPushMatrix();
        glTranslatef(-15.5, 4.5, z);
        glScalef(0.2, 0.8, 0.2);
        glutSolidCube(1);
        glPopMatrix();

        // direita
        glPushMatrix();
        glTranslatef(15.5, 4.5, z);
        glScalef(0.2, 0.8, 0.2);
        glutSolidCube(1);
        glPopMatrix();

        // lampadas
        GLfloat emissive[] = {1.5, 1.1, 0.5, 1.0};
        glMaterialfv(GL_FRONT, GL_EMISSION, emissive);

        glColor3f(1.0, 0.75, 0.3);

        // esquerda
        glPushMatrix();
        glTranslatef(-15.5, 4.2, z);
        glutSolidSphere(0.18, 16, 16);
        glPopMatrix();

        // direita
        glPushMatrix();
        glTranslatef(15.5, 4.2, z);
        glutSolidSphere(0.18, 16, 16);
        glPopMatrix();

        // desliga emissão
        GLfloat no_emissive[] = {0.0, 0.0, 0.0, 1.0};
        glMaterialfv(GL_FRONT, GL_EMISSION, no_emissive);
    }
}

void desenharCorredorLateral()
{
    glColor3f(0.5, 0.08, 0.15);
    glPushMatrix();
    glTranslatef(11, -0.2, 4);
    glScalef(5.5, 0.05, 48);

```

```

glutSolidCube(1);
glPopMatrix();

glColor3f(0.85, 0.72, 0.35);
for (float z = -18; z <= 25; z += 4)
{
    glPushMatrix();
    glTranslatef(11.5, -0.1, z);
    glScalef(0.35, 0.05, 0.35);
    glutSolidCube(1);
    glPopMatrix();
}

void desenharaEscadaria()
{
    float altura = 0;
    for (float z = -18; z <= 25; z += 3.2)
    {
        glColor3f(0.22, 0.22, 0.28);
        glPushMatrix();
        glTranslatef(11, altura - 0.15, z);
        glScalef(4.5, 0.2, 2.8);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();

        glColor3f(0.5, 0.08, 0.15);
        glPushMatrix();
        glTranslatef(11, altura - 0.05, z);
        glScalef(4.3, 0.05, 2.6);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();

        altura += 0.18;
    }
}

void desenharaPorta()
{
    glColor3f(0.35, 0.18, 0.08);
    glPushMatrix();
    glTranslatef(11, 1.8, 26);
    glScalef(2.2, 4.0, 0.1);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.85, 0.72, 0.35);
    glPushMatrix();
    glTranslatef(10.2, 2.0, 26.05);
    glutSolidSphere(0.1, 12, 12);
    glPopMatrix();

    glColor3f(0.8, 0.6, 0.1);
    glPushMatrix();
    glTranslatef(11, 3.2, 26.05);
    glScalef(1.2, 0.3, 0.05);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.08, 0.06, 0.12);
    glPushMatrix();
    glTranslatef(13.5, 2.5, 26);
    glScalef(0.2, 5.0, 0.5);
    glutSolidCube(1);
    glPopMatrix();
}

```

```

void desenharCaixasSom()
{
    glColor3f(0.22, 0.22, 0.28);

    glPushMatrix();
    glTranslatef(-10, 4.5, -21);
    glScalef(1.2, 2.5, 1.0);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.0, 0.0, 0.0);
    glPushMatrix();
    glTranslatef(-10, 4.0, -20.7);
    glutSolidSphere(0.4, 20, 20);
    glPopMatrix();

    glColor3f(0.22, 0.22, 0.28);
    glPushMatrix();
    glTranslatef(10, 4.5, -21);
    glScalef(1.2, 2.5, 1.0);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.0, 0.0, 0.0);
    glPushMatrix();
    glTranslatef(10, 4.0, -20.7);
    glutSolidSphere(0.4, 20, 20);
    glPopMatrix();
}

void desenharParedes()
{
    glColor3f(0.08, 0.06, 0.12);

    // parede frontal (tela)
    glPushMatrix();
    glTranslatef(0, 4.5, -24);
    glScalef(32, 9.0, 0.2);
    glNormal3f(0, 0, 1);
    glutSolidCube(1);
    glPopMatrix();

    // parede esquerda
    glPushMatrix();
    glTranslatef(-16, 4.5, 0);
    glScalef(0.2, 9.0, 55);
    glNormal3f(1, 0, 0);
    glutSolidCube(1);
    glPopMatrix();

    // parede direita
    glPushMatrix();
    glTranslatef(16, 4.5, 0);
    glScalef(0.2, 9.0, 55);
    glNormal3f(-1, 0, 0);
    glutSolidCube(1);
    glPopMatrix();

    // parede traseira
    glPushMatrix();
    glTranslatef(0, 4.5, 28);
    glScalef(32, 9.0, 0.2);
    glNormal3f(0, 0, -1);
    glutSolidCube(1);
    glPopMatrix();
}

```

```
// Ana Vitória Cardoso Queiroz - 172706
// Maria Clara Muharem - 266864
```

```
#include "obj.h"
#include <GL/glut.h>
#include <math.h>

void desenharCadeiraVIP()
{
    glPushMatrix();

    // base
    glColor3f(0.85, 0.72, 0.35);
    glPushMatrix();
    glTranslatef(0, 0.08, 0);
    glScalef(1.8, 0.08, 1.6);
    glNormal3f(0, 1, 0);
    glutSolidCube(1.0);
    glPopMatrix();

    // assento
    glColor3f(0.45, 0.25, 0.18);
    glPushMatrix();
    glTranslatef(0, 0.35, 0.1);
    glScalef(1.7, 0.25, 1.5);
    glNormal3f(0, 1, 0);
    glutSolidCube(1.0);
    glPopMatrix();

    // assento acabamento
    glColor3f(0.55, 0.18, 0.22);
    glPushMatrix();
    glTranslatef(0, 0.5, 0.1);
    glScalef(1.65, 0.1, 1.45);
    glNormal3f(0, 1, 0);
    glutSolidCube(1.0);
    glPopMatrix();

    // encosto
    glColor3f(0.45, 0.25, 0.18);
    glPushMatrix();
    glTranslatef(0, 0.9, 0.6);
    glRotatef(18, 1, 0, 0);
    glScalef(1.7, 1.3, 0.28);

    // Normal para face inclinada
    glNormal3f(0, 0.95, 0.31);
    glutSolidCube(1.0);
    glPopMatrix();

    // encosto
    glColor3f(0.55, 0.18, 0.22);
    glPushMatrix();
    glTranslatef(0, 0.9, 0.58);
    glRotatef(18, 1, 0, 0);
    glScalef(1.65, 1.25, 0.1);
    glNormal3f(0, 0.95, 0.31);
    glutSolidCube(1.0);
    glPopMatrix();

    // apoio cabeca
    glColor3f(0.45, 0.25, 0.18);
    glPushMatrix();
    glTranslatef(0, 1.45, 0.75);
    glRotatef(10, 1, 0, 0);
    glScalef(1.2, 0.35, 0.2);
    glNormal3f(0, 0.98, 0.17);
    glutSolidCube(1.0);
```

```

glPopMatrix();

// apoio braço esquerdo
glColor3f(0.35, 0.18, 0.08);
glPushMatrix();
glTranslatef(-0.95, 0.55, 0.05);
glScalef(0.18, 0.35, 1.5);
glNormal3f(-1, 0, 0);
glutSolidCube(1.0);
glPopMatrix();

// apoio braço esquerdo
glColor3f(0.55, 0.18, 0.22);
glPushMatrix();
glTranslatef(-0.95, 0.75, 0.05);
glScalef(0.18, 0.12, 1.45);
glNormal3f(-1, 0, 0);
glutSolidCube(1.0);
glPopMatrix();

// apoio braço direito (base)
glColor3f(0.35, 0.18, 0.08);
glPushMatrix();
glTranslatef(0.95, 0.55, 0.05);
glScalef(0.18, 0.35, 1.5);
glNormal3f(1, 0, 0);
glutSolidCube(1.0);
glPopMatrix();

// apoio braço direito (acabamento)
glColor3f(0.55, 0.18, 0.22);
glPushMatrix();
glTranslatef(0.95, 0.75, 0.05);
glScalef(0.18, 0.12, 1.45);
glNormal3f(1, 0, 0);
glutSolidCube(1.0);
glPopMatrix();
}

void desenharDegrau(float zPos, float yPos, float largura)
{
    // base do degrau
    glColor3f(0.22, 0.22, 0.28);
    glPushMatrix();
    glTranslatef(0, yPos - 0.1, zPos);
    glScalef(largura, 0.3, 3.2);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();

    // piso do degrau
    glColor3f(0.5, 0.08, 0.15);
    glPushMatrix();
    glTranslatef(0, yPos + 0.05, zPos);
    glScalef(largura - 0.5, 0.05, 3.0);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();

    // borda dourada
    glColor3f(0.85, 0.72, 0.35);
    glPushMatrix();
    glTranslatef(0, yPos + 0.18, zPos + 1.55);
    glScalef(largura - 0.8, 0.03, 0.08);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();
}

```

```

void desenharPlateia()
{
    int fileiras = 8;
    float zInicio = -3.0;
    float espacamentoZ = 3.8;
    float alturaPorDegrau = 0.65;

    for (int i = 0; i < fileiras; i++)
    {
        float zPos = zInicio + (i * espacamentoZ);
        float yPos = i * alturaPorDegrau;

        float larguraEsquerda = 9.5;
        desenharDegrau(zPos, yPos, larguraEsquerda);

        glPushMatrix();
        glTranslatef(11.5, 0, 0);
        desenharDegrau(zPos, yPos, larguraEsquerda);
        glPopMatrix();

        for (float x = -7.5; x <= -2.5; x += 2.2)
        {
            glPushMatrix();
            glTranslatef(x, yPos + 0.25, zPos);
            desenharCadeiraVIP();
            glPopMatrix();
        }

        for (float x = 3.5; x <= 8.5; x += 2.2)
        {
            glPushMatrix();
            glTranslatef(x, yPos + 0.25, zPos);
            desenharCadeiraVIP();
            glPopMatrix();
        }

        if (i >= 5)
        {
            for (float x = -10; x <= -8; x += 2.0)
            {
                glPushMatrix();
                glTranslatef(x, yPos + 0.25, zPos);
                desenharCadeiraVIP();
                glPopMatrix();
            }
            for (float x = 9; x <= 11; x += 2.0)
            {
                glPushMatrix();
                glTranslatef(x, yPos + 0.25, zPos);
                desenharCadeiraVIP();
                glPopMatrix();
            }
        }
    }
}

```

```

void desenharChao()
{
    glColor3f(0.22, 0.22, 0.28);
    glPushMatrix();
    glTranslatef(0, -0.3, 0);
    glScalef(32, 0.1, 55);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.5, 0.08, 0.15);
}

```

```

    glPushMatrix();
    glTranslatef(0, -0.25, 0);
    glScalef(30, 0.05, 53);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();
}

void desenharaTeto()
{
    glColor3f(0.08, 0.06, 0.12);
    glPushMatrix();
    glTranslatef(0, 9.0, 0);
    glScalef(32, 0.15, 55);
    glNormal3f(0, -1, 0);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.85, 0.72, 0.35);
    for (float x = -15; x <= 15; x += 30)
    {
        glPushMatrix();
        glTranslatef(x, 8.6, 0);
        glScalef(0.3, 0.4, 54);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();
    }

    glColor3f(1.0, 0.9, 0.6);

    for (float x = -11; x <= -3; x += 2.0)
    {
        for (float z = -20; z <= 25; z += 4.0)
        {
            glPushMatrix();
            glTranslatef(x, 8.92, z);
            glScalef(0.8, 0.03, 1.2);
            glNormal3f(0, 1, 0);
            glutSolidCube(1);
            glPopMatrix();
        }
    }

    for (float x = 4; x <= 12; x += 2.0)
    {
        for (float z = -20; z <= 25; z += 4.0)
        {
            glPushMatrix();
            glTranslatef(x, 8.92, z);
            glScalef(0.8, 0.03, 1.2);
            glNormal3f(0, 1, 0);
            glutSolidCube(1);
            glPopMatrix();
        }
    }

    for (float z = -18; z <= 27; z += 3.5)
    {
        glPushMatrix();
        glTranslatef(0, 8.92, z);
        glScalef(1.5, 0.03, 1.0);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();
    }

    for (float z = -18; z <= 27; z += 2.5)

```

```

{
    glPushMatrix();
    glTranslatef(11, 8.92, z);
    glScalef(0.6, 0.03, 1.8);
    glNormal3f(0, 1, 0);
    glutSolidCube(1);
    glPopMatrix();
}

for (float x = -11; x <= -3; x += 2.0)
{
    for (float z = -20; z <= 25; z += 4.0)
    {
        glPushMatrix();
        glTranslatef(x, 8.89, z);
        glScalef(0.9, 0.02, 1.3);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();
    }
}

for (float x = 4; x <= 12; x += 2.0)
{
    for (float z = -20; z <= 25; z += 4.0)
    {
        glPushMatrix();
        glTranslatef(x, 8.89, z);
        glScalef(0.9, 0.02, 1.3);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();
    }
}

// projetor
glColor3f(0.3, 0.3, 0.35);
glPushMatrix();
glTranslatef(0, 8.2, -15);
glScalef(1.5, 0.3, 1.8);
glNormal3f(0, 1, 0);
glutSolidCube(1);
glPopMatrix();

glColor3f(0.1, 0.1, 0.15);
glPushMatrix();
glTranslatef(0, 8.0, -14.5);
glScalef(1.2, 0.2, 0.8);
glNormal3f(0, 1, 0);
glutSolidCube(1);
glPopMatrix();

glColor3f(0.5, 0.5, 0.6);
glPushMatrix();
glTranslatef(0, 7.9, -14.8);
glScalef(0.8, 0.1, 0.5);
glNormal3f(0, 1, 0);
glutSolidCube(1);
glPopMatrix();
}

void desenharTela()
{
    // moldura externa
    glColor3f(0.85, 0.72, 0.35);
    glPushMatrix();
    glTranslatef(0, 5.0, -23.5);
    glScalef(12, 8.0, 0.1);

```



```

glutSolidCube(1);
glPopMatrix();

GLfloat emissive[] = {0.2, 0.2, 0.2, 1.0}; // leve brilho
glMaterialfv(GL_FRONT, GL_EMISSION, emissive);

glColor3f(0.95, 0.95, 0.95);
glPushMatrix();
glTranslatef(0, 5.0, -23.3);
glScalef(11.5, 7.5, 0.05);
glutSolidCube(1);
glPopMatrix();

// desliga emissão
GLfloat no_emissive[] = {0.0, 0.0, 0.0, 1.0};
glMaterialfv(GL_FRONT, GL_EMISSION, no_emissive);

// laterais
glColor3f(0.65, 0.12, 0.15);
glPushMatrix();
glTranslatef(-6.5, 5.0, -23);
glScalef(1.2, 8.0, 0.4);
glutSolidCube(1);
glPopMatrix();

glPushMatrix();
glTranslatef(6.5, 5.0, -23);
glScalef(1.2, 8.0, 0.4);
glutSolidCube(1);
glPopMatrix();

// topo
glPushMatrix();
glTranslatef(0, 8.8, -23);
glScalef(13, 0.8, 0.5);
glutSolidCube(1);
glPopMatrix();
}

void desenharLuzes()
{
    for (float z = -20; z <= 25; z += 3.5)
    {
        glColor3f(0.6, 0.5, 0.3);

        // esquerda
        glPushMatrix();
        glTranslatef(-15.5, 4.5, z);
        glScalef(0.2, 0.8, 0.2);
        glutSolidCube(1);
        glPopMatrix();

        // direita
        glPushMatrix();
        glTranslatef(15.5, 4.5, z);
        glScalef(0.2, 0.8, 0.2);
        glutSolidCube(1);
        glPopMatrix();

        // lampadas
        GLfloat emissive[] = {1.5, 1.1, 0.5, 1.0};
        glMaterialfv(GL_FRONT, GL_EMISSION, emissive);

        glColor3f(1.0, 0.75, 0.3);

        // esquerda

```

```

    glPushMatrix();
    glTranslatef(-15.5, 4.2, z);
    glutSolidSphere(0.18, 16, 16);
    glPopMatrix();

    // direita
    glPushMatrix();
    glTranslatef(15.5, 4.2, z);
    glutSolidSphere(0.18, 16, 16);
    glPopMatrix();

    // desliga emissão
    GLfloat no_emissive[] = {0.0, 0.0, 0.0, 1.0};
    glMaterialfv(GL_FRONT, GL_EMISSION, no_emissive);
}
}

```

```

void desenharCorredorLateral()
{
    glColor3f(0.5, 0.08, 0.15);
    glPushMatrix();
    glTranslatef(11, -0.2, 4);
    glScalef(5.5, 0.05, 48);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.85, 0.72, 0.35);
    for (float z = -18; z <= 25; z += 4)
    {
        glPushMatrix();
        glTranslatef(11.5, -0.1, z);
        glScalef(0.35, 0.05, 0.35);
        glutSolidCube(1);
        glPopMatrix();
    }
}

```

```

void desenharEscadaria()
{
    float altura = 0;
    for (float z = -18; z <= 25; z += 3.2)
    {
        glColor3f(0.22, 0.22, 0.28);
        glPushMatrix();
        glTranslatef(11, altura - 0.15, z);
        glScalef(4.5, 0.2, 2.8);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();

        glColor3f(0.5, 0.08, 0.15);
        glPushMatrix();
        glTranslatef(11, altura - 0.05, z);
        glScalef(4.3, 0.05, 2.6);
        glNormal3f(0, 1, 0);
        glutSolidCube(1);
        glPopMatrix();

        altura += 0.18;
    }
}

```

```

void desenharPorta()
{
    glColor3f(0.35, 0.18, 0.08);
    glPushMatrix();
    glTranslatef(11, 1.8, 26);
    glScalef(2.2, 4.0, 0.1);
}

```

```

glutSolidCube(1);
glPopMatrix();

glColor3f(0.85, 0.72, 0.35);
glPushMatrix();
glTranslatef(10.2, 2.0, 26.05);
glutSolidSphere(0.1, 12, 12);
glPopMatrix();

glColor3f(0.8, 0.6, 0.1);
glPushMatrix();
glTranslatef(11, 3.2, 26.05);
glScalef(1.2, 0.3, 0.05);
glutSolidCube(1);
glPopMatrix();

glColor3f(0.08, 0.06, 0.12);
glPushMatrix();
glTranslatef(13.5, 2.5, 26);
glScalef(0.2, 5.0, 0.5);
glutSolidCube(1);
glPopMatrix();
}

void desenharCaixasSom()
{
    glColor3f(0.22, 0.22, 0.28);

    glPushMatrix();
    glTranslatef(-10, 4.5, -21);
    glScalef(1.2, 2.5, 1.0);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.0, 0.0, 0.0);
    glPushMatrix();
    glTranslatef(-10, 4.0, -20.7);
    glutSolidSphere(0.4, 20, 20);
    glPopMatrix();

    glColor3f(0.22, 0.22, 0.28);
    glPushMatrix();
    glTranslatef(10, 4.5, -21);
    glScalef(1.2, 2.5, 1.0);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.0, 0.0, 0.0);
    glPushMatrix();
    glTranslatef(10, 4.0, -20.7);
    glutSolidSphere(0.4, 20, 20);
    glPopMatrix();
}

void desenharParedes()
{
    glColor3f(0.08, 0.06, 0.12);

    // parede frontal (tela)
    glPushMatrix();
    glTranslatef(0, 4.5, -24);
    glScalef(32, 9.0, 0.2);
    glNormal3f(0, 0, 1);
    glutSolidCube(1);
    glPopMatrix();

    // parede esquerda
    glPushMatrix();

```

```

glTranslatef(-16, 4.5, 0);
glScalef(0.2, 9.0, 55);
glNormal3f(1, 0, 0);
glutSolidCube(1);
glPopMatrix();

// parede direita
glPushMatrix();
glTranslatef(16, 4.5, 0);
glScalef(0.2, 9.0, 55);
glNormal3f(-1, 0, 0);
glutSolidCube(1);
glPopMatrix();

// parede traseira
glPushMatrix();
glTranslatef(0, 4.5, 28);
glScalef(32, 9.0, 0.2);
glNormal3f(0, 0, -1);
glutSolidCube(1);
glPopMatrix();
}

```

textura.c

C/C++

```

// Ana Vitória Cardoso Queiroz - 172706
// Maria Clara Muharem - 266864

```

```

#include "textura.h"
#include "image.h"
#include "obj.h"
#include <stdio.h>
#include <GL/glut.h>

```

```

GLuint textureIDParede = 0;
GLuint textureIDChao = 0;
int texturasAtivas = 1;
IMAGE *textureImage = NULL;

```

```

void carregarTexturas() {

```

```

    glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE);
    //parede
    textureImage = ImageLoad("wall.rgb");
    {
        glGenTextures(1, &textureIDParede);
        glBindTexture(GL_TEXTURE_2D, textureIDParede);

        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    }

```

```

    if (textureImage->sizeZ == 3) {
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureImage->sizeX,
            textureImage->sizeY, 0, GL_RGB, GL_UNSIGNED_BYTE,
            textureImage->data);
    } else if (textureImage->sizeZ == 4) {
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, textureImage->sizeX,
            textureImage->sizeY, 0, GL_RGBA, GL_UNSIGNED_BYTE,
            textureImage->data);
    }

```

```

    free(textureImage->data);

```

```

    free(textureImage);
    textureImage = NULL;
}
//chao
textureImage = ImageLoad("chao.rgb");
{
    glGenTextures(1, &textureIDChao);
    glBindTexture(GL_TEXTURE_2D, textureIDChao);

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    if (textureImage->sizeZ == 3) {
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureImage->sizeX,
            textureImage->sizeY, 0, GL_RGB, GL_UNSIGNED_BYTE,
            textureImage->data);
    } else if (textureImage->sizeZ == 4) {
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, textureImage->sizeX,
            textureImage->sizeY, 0, GL_RGBA, GL_UNSIGNED_BYTE,
            textureImage->data);
    }

    free(textureImage->data);
    free(textureImage);
    textureImage = NULL;
}
}

void toggleTexturas() {
    texturasAtivas = !texturasAtivas;
    glutPostRedisplay();
}

//função com normal específica para cada parede
void aplicarTexturaNaParede(float x1, float y1, float z1, float x2, float y2, float z2, float repX, float repY, int paredeTipo) {
    glBegin(GL_QUADS);

    // Define a normal baseada no tipo da parede
    switch(paredeTipo) {
        case 0: // parede frontal (tela) - aponta para +Z
            glNormal3f(0.0, 0.0, 1.0); break;
        case 1: // parede esquerda - aponta para +X
            glNormal3f(1.0, 0.0, 0.0); break;
        case 2: // parede direita - aponta para -X
            glNormal3f(-1.0, 0.0, 0.0); break;
        case 3: // parede traseira - aponta para -Z
            glNormal3f(0.0, 0.0, -1.0); break;
    }

    glTexCoord2f(0.0f, 0.0f); glVertex3f(x1, y1, z1);
    glTexCoord2f(repX, 0.0f); glVertex3f(x2, y1, z2);
    glTexCoord2f(repX, repY); glVertex3f(x2, y2, z2);
    glTexCoord2f(0.0f, repY); glVertex3f(x1, y2, z1);
    glEnd();
}

void aplicarTexturaNoChao(float x1, float y1, float z1, float x2, float y2, float z2, float repX, float repZ) {
    glNormal3f(0.0, 1.0, 0.0);
    glBegin(GL_QUADS);
    glTexCoord2f(0.0f, 0.0f); glVertex3f(x1, y1, z1);
    glTexCoord2f(repX, 0.0f); glVertex3f(x2, y1, z1);
    glTexCoord2f(repX, repZ); glVertex3f(x2, y2, z2);
    glTexCoord2f(0.0f, repZ); glVertex3f(x1, y2, z2);
    glEnd();
}
}

```

```

void desenharParedesComTextura() {
    if (texturasAtivas && textureIDParede != 0) {
        glEnable(GL_TEXTURE_2D);
        glBindTexture(GL_TEXTURE_2D, textureIDParede);
        glColor3f(1.0f, 1.0f, 1.0f);

        // parede tela (frontal) - tipo 0
        aplicarTexturaNaParede(-16, 0, -24, 16, 9, -24, 4.0f, 3.0f, 0);
        // parede esquerda - tipo 1
        aplicarTexturaNaParede(-16, 0, -24, -16, 9, 28, 8.0f, 3.0f, 1);
        // parede direita - tipo 2
        aplicarTexturaNaParede(16, 0, -24, 16, 9, 28, 8.0f, 3.0f, 2);
        // parede tras - tipo 3
        aplicarTexturaNaParede(-16, 0, 28, 16, 9, 28, 4.0f, 3.0f, 3);

        glDisable(GL_TEXTURE_2D);
    } else {
        desenharParedes();
    }
}

void desenharChaoComTextura() {
    if (texturasAtivas && textureIDChao != 0) {
        glEnable(GL_TEXTURE_2D);
        glBindTexture(GL_TEXTURE_2D, textureIDChao);
        glColor3f(1.0f, 1.0f, 1.0f);

        aplicarTexturaNoChao(-16, -0.3, -24, 16, -0.3, 28, 8.0f, 13.0f);

        glDisable(GL_TEXTURE_2D);
    } else {
        desenharChao();
    }
}

```

Imagens: O intuito é a luz da tela iluminar as cadeiras.



